

Developer Guide

Prerequisites

- Node.js compatible with the locked dependencies (use a current LTS release and record it in `.nvmrc` for repeatability).
- npm.
- MongoDB running locally or a MongoDB connection string.
- Two terminals for client and server development.

Local setup

```
git clone https://github.com/karthikv1711/Darshan-Ease.git
cd Darshan-Ease

cd server
npm install

cd ../client
npm ci
```

Create `server/.env`:

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/darshanease
JWT_SECRET=replace-with-a-long-random-development-secret
```

Start the backend:

```
cd server
npm run dev
```

Start the frontend in another terminal:

```
cd client
npm run dev
```

Open the URL printed by Vite (normally `http://localhost:5173`). The repository's existing README incorrectly suggests `npm start` and port 3000 for the frontend; the actual client scripts use `npm run dev` and Vite.

Startup behavior

1. The backend loads `.env` and calls `connectDB()` without awaiting it before listening.
2. API requests are accepted by Express, but `/api` returns 503 until Mongoose is connected.
3. After connection, the seeder runs.
4. If no temples exist, it inserts three temples and twelve slots (four per temple).
5. If temples exist, it upgrades known seed temples and adds missing slots for them.

Seeding is not fully deterministic: event and slot dates are relative to startup, and update/upsert behavior may create new known temple documents without restoring every embedded sample event/rating.

Project layout

```
Darshan-Ease/
├── client/
│   ├── public/           # static images, audio, icons
│   ├── src/components/   # navbar, footer, audio, modals
│   ├── src/context/      # auth and audio state
│   ├── src/pages/        # public pages and dashboards
│   ├── package.json
│   └── vite.config.js
└── server/
    ├── config/           # MongoDB connection and seed
    ├── controllers/      # HTTP/business logic
    ├── middleware/       # JWT and role checks
    ├── models/           # Mongoose schemas
    ├── routes/           # Express route declarations
    ├── package.json
    └── server.js
```

Client development notes

- `AuthContext` globally sets Axios to `http://localhost:5000/api`. Replace this with `import.meta.env.VITE_API_BASE_URL` before deployment.
- JWT is stored in `localStorage`, which is simple but exposes it to successful XSS. Consider a hardened cookie/session design.
- Dashboard route elements are not protected. Add a route guard for navigation/UX, but never treat client guards as authorization.
- Most styling is in `App.css` and `index.css`, with extensive inline style objects.
- The booking and donation card checks are browser-only simulation logic; no card values are included in API payloads.
- Organizer notifications are a static in-memory array in `DashboardOrganizer.jsx`.

Server development notes

- Controllers directly contain business logic and database access. A service layer would make state-transition testing easier.

- Most controller failures are returned as 500, including invalid ObjectIds and validation errors. Normalize these into 400/404/409 as appropriate.
- Always enforce business rules in the server even if the UI validates them.
- Use an async error wrapper to reduce repeated `try/catch` blocks.
- Do not commit `server/node_modules`; keep lockfiles and use `npm ci`.

Common commands

Module	Command	Purpose
Client	<code>npm run dev</code>	Vite development server
Client	<code>npm run build</code>	Production bundle
Client	<code>npm run lint</code>	Oxlint checks
Client	<code>npm run preview</code>	Preview built frontend
Server	<code>npm start</code>	Start Express
Server	<code>npm run dev</code>	Currently same as start; no watcher

Environment configuration recommended for production

Variable	Required	Example/purpose
<code>NODE_ENV</code>	Yes	<code>production</code>
<code>PORT</code>	Yes	Backend listen port
<code>MONGODB_URI</code>	Yes	Authenticated TLS MongoDB URI
<code>JWT_SECRET</code>	Yes	Long random secret from a secret manager
<code>JWT_EXPIRES_IN</code>	Recommended	Avoid hard-coded 30 days
<code>CORS_ORIGINS</code>	Yes	Explicit trusted frontend origins
<code>VITE_API_BASE_URL</code>	Client build	Public API origin/base path

Contribution workflow

Use a short-lived branch, keep changes scoped, add tests with every behavior change, run client build/lint and backend tests locally, document API/schema changes, and use pull-request review. Database index or schema changes need a rollout and rollback note.